

Creating Delegates, Events, and EventArgs



Dan Wahlin

Wahlin Consulting

@DanWahlin | www.codewithdan.com



Overview



Creating a Delegate

Defining an Event

Raising Events

Creating an EventArgs Class





Creating a Delegate



Event Handler

EventArgs

Event Raiser

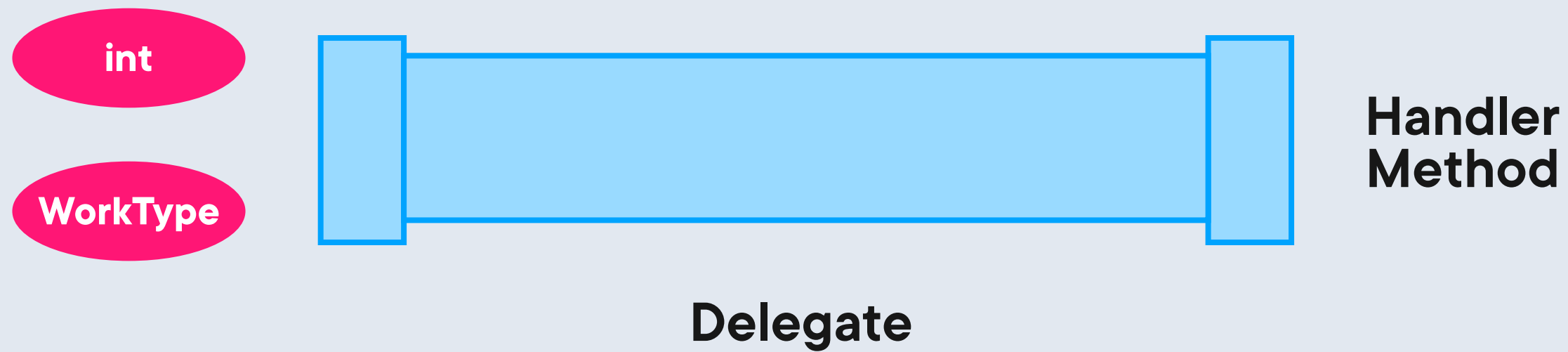
Delegate





Delegate

```
public delegate int WorkPerformedHandler(int hours, WorkType workType);
```



Creating a Delegate

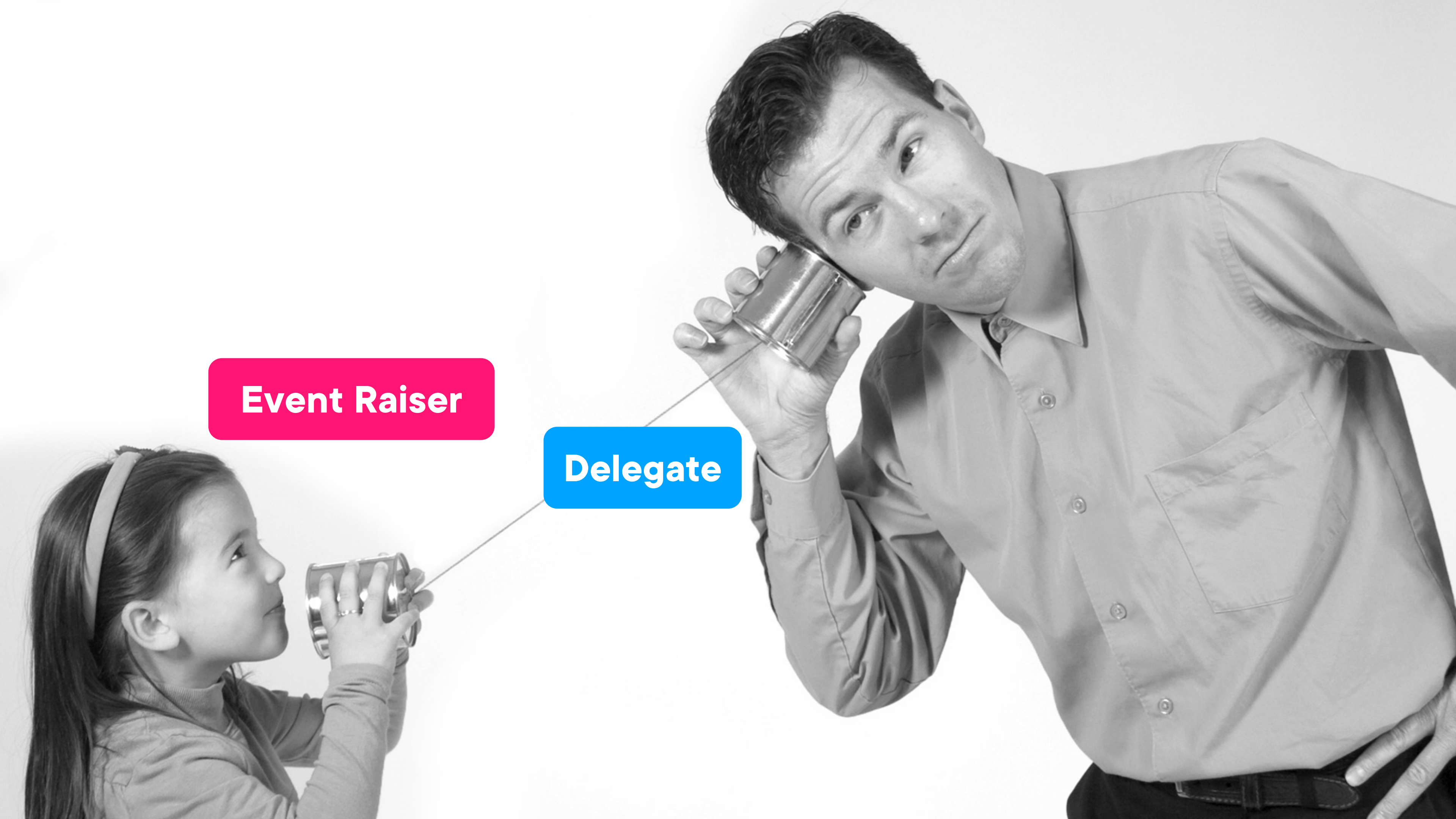
Custom delegates are defined using the **delegate** keyword.





Defining an Event





Event Raiser

Delegate

```
public event WorkPerformedHandler? WorkPerformed;
```

Defining an Event

Events can be defined in a class using the **event** keyword.





Raising Events



Event Handler



```
public event WorkPerformedHandler? WorkPerformed;
```

```
...
```

```
if (WorkPerformed != null) {
```

```
    WorkPerformed(8, WorkType.GenerateReports);
```

```
}
```

Raising Events

Events are raised by calling the event like a method.



```
public event WorkPerformedHandler? WorkPerformed;
```

```
...
```

```
WorkPerformed?.Invoke(8, WorkType.GenerateReports);
```

Using Invoke() to Raise an Event

The Invoke() method can be used to simplify the code required to raise an event.



Exposing and Raising Events

```
public delegate void WorkPerformedHandler(int hours, WorkType workType);

public class Worker
{
    public event WorkPerformedHandler? WorkPerformed;
    public void DoWork(int hours, WorkType workType)
    {
        // Do work here and notify consumer that work has been performed
        OnWorkPerformed(hours, workType);
    }

    protected virtual void OnWorkPerformed(int hours, WorkType workType)
    {
        WorkPerformed?.Invoke(hours, WorkType.GenerateReports);
    }
}
```



Creating an EventArgs Class



Event Handler

EventArgs

Event Raiser

Delegate



```
public void Button_Click(object sender, EventArgs e)
{
    // Handle button click
}
```

The EventArgs Class

The EventArgs class is used in the signature of many delegates and event handlers.



```
public class WorkPerformedEventArgs
{
    public int Hours { get; }
    public WorkType WorkType { get; }

    public WorkPerformedEventArgs(int hours, WorkType workType) =>
        (Hours, WorkType) = (hours, workType);
}
```

Creating a Custom EventArgs Class

An EventArgs class defines properties that hold data that is sent when the event is raised.



```
public delegate void WorkPerformedHandler(object sender,  
    WorkPerformedEventArgs e);
```

Using a Custom EventArgs Class

To use a custom EventArgs class, the delegate must reference the class in its signature.



```
public delegate void WorkPerformedEventHandler(object sender,  
    WorkPerformedEventArgs e);
```

```
public event EventHandler<WorkPerformedEventArgs>? WorkPerformed;
```

Using EventHandler<T>

.NET includes a generic **EventHandler<T>** class that can be used instead of a custom delegate.



Summary



Events are associated with Delegates

A standard event signature accepts:

- Sender
- EventArgs

The parameter signature for an event handler matches the delegate signature

EventHandler<T> provides a simple way to create a custom delegate for an event

Up Next:

Handling Events

